

计算机程序设计 · 2025 年真题（题面还原 + 详解）

类型：题面来自真题讲解视频还原（B站 UP「一灯computer」· [BV1Z4w3znE6h](#) P1-P4）

满分 200 分 · 150 分钟

科目：广东普通专升本《计算机基础与程序设计》

边界：考试院不公开发布原卷 PDF。本页题面由[视频画面帧逐题核对](#)还原，选项文字与答案以帧为准；
措辞细节仍以你手头正版真题册为准。见 [00-资料来源与使用说明](#)

按考点拆练：[考点拆分索引](#)

★ 本页的定位（与另一页的分工）：

| 页面 | 内容 | 资料等级 |

|:---|:---|:---|

| 本页 | 真真题面（视频逐题讲解还原）+ 详解 | B 级（题面有帧为证） |

| [2025 真题回忆版详解](#) | 考生回忆碎片 + 同型重构题（45 题里 41 题标注  非原题） | C 级 |

| 本页旧版（已替换） | 只有官方题型结构，题面写「待补充」 | 一 |

两页互补，不重复：要原题面看本页；要更多同型练习题看回忆版。

还原方法见 [B站吸收规划·计算机](#)。

试卷结构

大题	题号	题量	分值	核心考查
一、单项选择题	1-20	20	60	C 语法、运算符、数据结构

二、判断题	21-30	10	20	C 语言细节、数据结构性质
三、填空题	31-35	5	20	数制与表示、二叉树性质、文件操作
四、简答题	36-39	4	40	概念辨析、程序分析（本页无源）
五、计算题	40-42	3	30	程序执行结果分析（本页无源）
六、应用题	43-45	3	30	手写 C 语言程序（本页无源）
合计		45	200	满分 200 分 · 150 分钟（本页已还原客观题 100 分）

来源：新东方在线《2025 年广东专升本计算机基础与程序设计考试题型》（2025-02-26 发布）。题型与 2024 完全相同。

题型	题量	分值	核心考查内容	本页状态
一、单项选择题	20	60（3 分/题）	C 语法基础、运算符、数据结构概念	✅ 已完成 20/20
二、判断题	10	20（2 分/题）	C 语言细节、数据结构性质	✅ 已完成 10/10
三、填空题	5	20（4 分/题）	数制与表示、二叉树性质、文件操作	✅ 已完成 5/5
四、简答题	4	40（10 分/题）	概念辨析、程序分析	❌ 无视频源
五、计算题	3	30（10 分/题）	程序执行结果分析	❌ 无视频源
六、应用题	3	30（10 分/题）	手写 C 语言程序	❌ 无视频源
合计		200		客观题 100 分已还原

官方参考教材（新东方口径 2025）：严蔚敏、李冬梅、吴伟民编著《数据结构（C 语言版·附微课视频）》（第 2 版），人民邮电出版社 2021。C 语言部分仍按谭浩强《C 语言程序设计》体系复习。

✅ 教材版本已定版（2026-08 官方考纲确认）：2026 官方考纲指定 严蔚敏、吴伟民《数据结构（C 语言版）》第 2 版，清华大学出版社 2025-10（ISBN 9787302703396）。备考购书认准清华社 2025-10 第 2 版。

题目一览

题号	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
答案	C	A	D	D	B	D	B	B	D	A	B	B	C	D	C	D	D	A

题号	19	20	21	22	23	24	25	26	27	28	29	30	31		32	33	34	35
答案	A	C	×	×	√	×	√	√	×	×	√	√	0.123456e5		1	0	0	(2504) ₈

一、单项选择题（本大题共 20 小题，每小题 3 分，共 60 分）

第 1 题 • sizeof 运算符

C语言中，可用来测定数据类型长度的运算符是（ ）

- A. &
- B. ==
- C. sizeof
- D. >>

答案与解析

答案：C · 考点 1.2 数据的存储与运算

解析：

- sizeof 是关键字，不是函数 —— 只是用法长得像函数（ sizeof(a) 的括号不是函数调用括号）。
- A & 是取地址，B == 是相等判断，D >> 是右移。
- 配套记忆：16 位机 int 占 2 字节，32/64 位机占 4 字节。真题考得较多的是 16 位机。

第 2 题 · 表达式求值

下列C语言表达式的值为2的是 ()

- A. 5%3
- B. 5/3
- C. 5>3 && 1>2
- D. a=2,b=8

答案与解析

答案：A · 考点 1.2 数据的存储与运算

解析：

选项	值	依据
A 5%3	2	求余。结果正负与被除数相同： $-5\%3 = -2$ 、 $5\%-3 = 2$
B 5/3	1	两侧都是整数 $\Rightarrow$ 结果必为整数，向下取整（不四舍五入）
C 5>3 && 1>2	0	$1 \ \&\& \ 0 = 0$ 。逻辑与两侧全真才为真
D a=2, b=8	8	逗号表达式取最后一个

⚠ 顺带一个常考类型题： $5/2.0$ 结果是 2.5，问类型要答 `double` —— 所有实型常量一律是 `double`。

第 3 题 · 条件表达式

已知 int a=2, b=3; 则条件表达式 a>b?1:0 的值是 ()

- A. 3
- B. 2

- C. 1
- D. 0

答案与解析

答案：D · 考点 1.4 选择结构程序设计

解析：`a>b` 即 `2>3`，**不成立** ⇒ 取冒号后面的 `0`。

三目运算符 条件 ? 真值 : 假值，共三个操作数，故称三目。

第 4 题 · strcat 与 strlen

已知 `char S1[10]="chatgpt"`, `S2[10]="deepseek"`, 则 `strlen(strcat(S1, S2))` 的值为 ()

- A. "deepseek"
- B. 8
- C. "chatgptdeepseek"
- D. 15

答案与解析

答案：D · 考点 1.6 数组（字符串处理函数）

解析：

- `strcat(S1, S2)`：把 `S2` **拼接**到 `S1` 后面，结果存回 `S1` ⇒ `"chatgptdeepseek"`
- `strlen(...)`：求长度 = `7 + 8 = 15`
- `strlen` **只算有效字符**，`'\0'` 不计入。（但**数组空间**要把 `'\0'` 算进去）

`string.h` **四个高频函数**（题中一次考了两个，另两个也要记）：

函数	作用	记法
<code>strcat</code>	拼接	concatenate
<code>strlen</code>	求长度	length
<code>strcpy</code>	复制	copy
<code>strcmp</code>	比较	compare, 返回负数/0/正数

⚠️ 两道送分题别丢：

- ① `char S1[10]; S1 = "abc";` **不合法** —— 数组名是**地址常量**，不能赋值。要改内容必须用 `strcpy`。
- ② 字符串字面量 `"abc"` 本身**也是地址**（字符 `'a'` 的地址），这是规定，记住即可。
- ③ `strcmp("abcd", "abcE")` ⇒ 逐字符比 ASCII，前三个相等，`'d' < 'E'` ⇒ 返回**负数**（默认 `-1`）。

⚠️ **真题原始写法提示：** `S1[10]` 实际装不下 15 个字符。**这是原卷的写法**，考的是 `strcat` 的语义而非数组容量 —— 别在这里纠结。

第 5 题 • 二叉树结点数

深度为4的二叉树结点数最多为（ ）

- A. 8
- B. 15
- C. 16
- D. 32

答案与解析

答案：B · 考点 2.5 树和二叉树

解析：

- 结点数**最多** $\Rightarrow$ 每层都填满 $\Rightarrow$ **满二叉树**
- 深度 4: $1 + 2 + 4 + 8 = 15 = 2^4 - 1$

两个必背公式：

问	公式
深度为 n 的 满 二叉树结点总数	$2^n - 1$
第 k 层的结点数（最多）	$2^{(k-1)}$

⚠ **满二叉树 vs 完全二叉树**（年年考区别）：

- **满二叉树**：每层都填满，非叶子结点都有两个子结点。
- **完全二叉树**：结点**从上到下、从左到右依次排列**，中间不能空。

判定口诀：「**只有右孩子、没有左孩子**」的结点一出现，就不是完全二叉树。

第 6 题 · 指针与数组

已知 `int a[] = {1,2,3,4}; int *p = a+1;` 则表达式 `*p * a[2]` 的值为 ()

- A. 2
- B. 3
- C. 4
- D. 6

答案与解析

答案：D · 考点 1.8 指针

解析：

下标	a[0]	a[1]	a[2]	a[3]
值	1	2	3	4
		↑		
		p = a+1		

- $p = a+1 \Rightarrow p$ 指向 $a[1] \Rightarrow *p = 2$
- $a[2] = 3$
- $*p * a[2] = 2 \times 3 = 6$

🔑 * 的两种身份（本题的真正考点）：

位置	含义
定义时 <code>int *p</code>	声明「p 是指针变量」
使用时 <code>*p</code>	解引用 （取值运算符）：取出 p 所指向内存里存的值

概念链：**指针就是地址，地址就是指针**；而**指针变量**是「用来存放地址的变量」。定义时类型必须与被指对象一致。

第 7 题 · 单链表插入

在单链表中，q 所指向结点为 p 所指结点的直接前驱结点，将 s 结点插入 q 和 p 之间的语句为（ ）

- A. $p \rightarrow next = s; s \rightarrow next = q$
- B. $q \rightarrow next = s; s \rightarrow next = p$
- C. $s \rightarrow next = p \rightarrow next; p \rightarrow next = s$
- D. $p \rightarrow next = s \rightarrow next; s \rightarrow next = p$

答案与解析

答案：B · 考点 2.2 线性表

解析：题设 q 在 p 前面，目标是把它变成 `q → s → p`。

插入前： `... → [q] → [p] → ...`

插入后： `... → [q] → [s] → [p] → ...`

- 让 q 的 next 指向 s: `q->next = s;`
- 让 s 的 next 指向 p: `s->next = p;`

两句先后顺序可以互换 —— 因为 `q->next` 和 `s->next` 互不依赖（不像「先断链再找后继」那种必须讲究顺序的写法）。

🔑 **单链表结点结构：** `data`（数据域）+ `next`（指针域，存下一个结点的地址）；最后一个结点 `next = NULL`。

直接前驱/后继 = 紧邻的前/后一个结点；不加「直接」二字时，前面的**所有**结点都叫前驱。

第 8 题 · 子串计数

串 "studio" 中，以 t 为首的子串个数为（ ）

- A. 2
- B. 5
- C. 6
- D. 7

答案与解析

答案：B · 考点 2.4 串、数组和广义表

解析：以 `t` 开头、连续向后取：

```
t
tu
tud
tudi
tudio
```

= 5 个。串本身也算子串（t 是单字符子串）。

✦ 本省对「串」考得不多且简单，不用花大力气。KMP 见 [2.4a 串与 KMP](#)。

第 9 题 · 广义表长度与深度

广义表 $L = (((a), b), c, d, e)$ 的长度和深度分别为（ ）

- A. 3和2
- B. 4和2
- C. 3和4
- D. 4和3

答案与解析

答案：D · 考点 2.4 串、数组和广义表

解析：

- 长度 = 最外层元素个数 = $((a), b), c, d, e = 4$
- 深度 = 括号最多嵌套几层 = 最外层 $\rightarrow ((a), b) \rightarrow (a) = 3$

🔑 广义表两条定义：

- 长度 = 最外层有几个元素（一个子表算一个元素，不算里面几个）

- 深度 = 数最深层的括号层数
- `L = ( )` 是空表，长度为 0

⚠ 该题括号写法以原卷为准 —— 视频画面帧上括号渲染不清，本页按「长度 4、深度 3」的答案反推结构。

第 10 题 · 树的形态数

一棵树有 3 个结点，其可能的形态个数为 ()

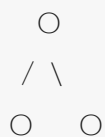
- A. 2
- B. 3
- C. 4
- D. 5

答案与解析

答案：A · 考点 2.5 树和二叉树

解析：树不分左右，3 个结点只有两种形态：

① 根挂两个



② 链式



= 2 种。

● 这是 2025 年考生集中做错的一道。讲课老师原话：

「去年考试的时候不少同学错了，选答案是 D 同学们在看题的时候一下没注意看，就选错了。」

改一个字，答案就变：

题面	答案	原因
树 有 3 个结点	A (2)	树 不区分左右 ，左子树/右子树同构视为同一种
二叉树 有 3 个结点	D (5)	二叉树 区分左右 ，共 5 种 (Catalan 数 $C_3 = 5$)

做题动作：看到「树/二叉树 有几个结点、几种形态」，先圈出是**树**还是**二叉树**。

第 11 题 · 复杂度增长

当 n

答案与解析

∞ 时，增长最快的是 ()

- A. $\log n$
- B. 2^n
- C. $n^2 \log n$
- D. $2n^2 - 1$

答案与解析

答案：B · 考点 2.9 算法基本概念与分析

解析：增长速度排序 (n 很大时)：

指数 2^n $\gg$ 幂 n^2 / $n^2 \log n$ $\gg$ 对数 $\log n$

指数函数增长最快 $\Rightarrow$ **B**。

✦ 这类题画函数图像最快，不用背表。见 [2.9 算法基本概念与分析](#)。

第 12 题 · 叙述正误

下列叙述正确的是（ ）

- **A.** 一个C语言程序可以有很多个main函数
- **B.** 一个C程序总是从main函数开始执行
- **C.** main函数必须放在程序的开始部分
- **D.** 一个C程序总是在第一个函数开始执行

答案与解析

答案：B · 考点 1.1 C语言概述与基本概念

解析：main 函数三条性质：

#	性质
1	一个 C 程序 有且只有一个 main 函数
2	main 函数的 位置任意 （不必放在最前或最后）
3	main 函数是程序的 入口和出口 （执行起点与终点）

A 错（只能有一个）、C 错（位置任意）、D 错（从 main 开始，不是第一个函数）。

第 13 题 · 复合赋值运算

程序段 `int a=6; a+=a-=a*a;` 执行后a的值为 ()

- A. 36
- B. 30
- C. -60
- D. -24

答案与解析

答案：C · 考点 1.2 数据的存储与运算

解析：赋值运算符从右至左结合，且优先级极低（倒数第二，只比逗号高），所以先算 `a*a`。

步	式子	计算	a 的值
0	—	初值	6
1	<code>a*a</code>	$6 \times 6 = 36$	6
2	<code>a -= 36</code>	$a = 6 - 36 = -30$	-30
3	<code>a += (-30)</code>	$a = -30 + (-30) = -60$	-60

复合赋值语义：`a += b` 等价于 `a = a + b`（左边的 a 不动，把 `a+b` 的结果赋回 a）。

⚠ 关键坑：第 2 步执行完 `a` 已经变成 -30，第 3 步用的是新值，不是原来的 6。很多人错在全程用 6。

第 14 题 · 逻辑表达式

正确表达逻辑关系"`a>10或a<0`"的 ()

- A. $a > 10$ or $a < 0$
- B. $a > 10 \mid a < 0$
- C. $a > 10 \ \&\& \ a < 10$
- D. $a > 10 \parallel a < 0$

答案与解析

答案：D · 考点 1.4 选择结构程序设计

解析：

选项	判读		
A	or 不是 C 运算符（那是 Python）		
B	\	\ 是按位或，不是逻辑或	
C	&& 是逻辑与，且 $a < 10$ 写错了		
D	\	\	\ 逻辑或 ✓

🔑 三个优先级（年年考，记住这一句就够）：

算术 > 关系 > 逻辑

例外：单目运算符优先级最高（! 逻辑非、~ 取反、++ / --）。

补充：赋值运算符倒数第二，逗号最低。

第 15 题 · 非线性结构

下列属于非线性数据结构的是（ ）

- A. 栈
- B. 队列

- C. 树
- D. 字符串

答案与解析

答案：C · 考点 2.1 数据结构基本概念

解析：

分类	包含
线性结构	表（顺序表 / 链表）、栈、队列、字符串
非线性结构	树、图、集合

⚠ 注意口径：问「从逻辑上可分为」答两类（线性 / 非线性）；问「基本逻辑结构有哪些」答四种（集合 / 线性 / 树形 / 图状）。

详见 [2.1 数据结构基本概念](#) 的「★ 考试口径」节。

另：存储（物理）结构只有两种 —— 顺序存储和链式存储。

第 16 题 · 无向图度数

一个无向连通图，有8个顶点，12条边，则所有顶点的度总和为（ ）

- A. 8
- B. 12
- C. 20
- D. 24

答案与解析

答案：D · 考点 2.6 图

解析：

所有顶点的度之和 = 边数 × 2

每条边连接两个顶点，给两端的度各贡献 1 $\Rightarrow 12 \times 2 = 24$ 。

 **与顶点个数无关！** 题里给的「8 个顶点」是干扰项 —— 这是本题的陷阱。

「无向」= 边没有方向；「连通」= 任意两顶点间都有路径。

第 17 题 · 二维数组访问

已知 `int a[][3] = { {0,1,2}, {3,4,5}, {6,7}, {8,9} }`；则 `a[2][1]` 的值是 ()

- A. 0
- B. 5
- C. 6
- D. 7

答案与解析

答案：D · 考点 1.6 数组

解析：不足的元素补 0：

行\列	[0]	[1]	[2]
a[0]	0	1	2
a[1]	3	4	5

a[2]	6	7	0
a[3]	8	9	0

`a[2][1] = 7。`

🔑 **二维数组初始化规则**（年年考）：

- **列号不能省，行号可以省** —— `int a[][3]` ✓ 合法；`int a[3][]` ✗ 不合法。
- **下标从 0 开始。**

第 18 题 · 运算符优先级

优先级最高的运算符是（ ）

- A. `~`
- B. `/`
- C. `<<`
- D. `&&`

答案与解析

答案：A · 考点 1.2 数据的存储与运算

解析：`~` 是**取反（按位取反）**，属**单目运算符**；单目运算符优先级最高。

🔑 **单目** = 只参与运算一个对象：`~a`、`!a`、`++a`、`a--`。

常规顺序：**算术（/） > 关系 > 逻辑（&&）**；`<<` 移位介于算术与关系之间。

做题动作：**先找哪个运算符只有一个操作数** —— 那个就是最高的。

第 19 题 • 标识符命名

C语言中不能为用户自定义标识符为（ ）

- A. auto
- B. _My
- C. boll
- D. done

答案与解析

答案：A · 考点 1.1 C语言概述与基本概念

解析：标识符三条规则：

#	规则
1	只能由 字母、数字、下划线 组成（\$、! 等一律不行）
2	不能以数字开头 （_abc ✓、2ab X）
3	不能是关键字

逐项判定：

选项	是什么	能否作标识符
A. auto	C 语言关键字（存储类别说明符）	✗ 不能 ← 答案
B. _My	下划线开头 + 字母，非关键字	✓ 能
C. boll	普通标识符，非关键字	✓ 能
D. done	普通标识符，非关键字	✓ 能

★ 题面按视频画面**原文照录**（原句是「不能为用户自定义标识符为」，通顺说法即「**不能用作**用户自定义标识符」）。

C 语言关键字全表（视频给了完整表格，老师圈出 auto）：

auto	break	case	char	const
continue	default	do	double	else
enum	extern	float	for	goto
if	inline	int	long	register
restrict	return	short	signed	sizeof
static	struct	switch	typedef	union
unsigned	void	volatile	while	_Bool
_Complex	_Imaginary			

记忆要点：

- sizeof 是关键字（不是函数）—— 本卷第 1 题就考这个。
- bool / true / false 不是关键字（它们是 <stdbool.h> 里的宏）；C99 的关键字是 _Bool。
- C99 在 C89 的 32 个关键字基础上新增 5 个：inline restrict _Bool _Complex _Imaginary。
- 选项 C 的 boll 疑似原卷 bool 的笔误，不影响答案（两者都不是关键字）。

第 20 题 • 循环队列判满

某循环队列Q的最大队列长度为M(M>3)，队头指针为front，队尾指针为rear，则Q的满队列条件为（ ）

- A. Q->front == Q->rear
- B. Q->front == Q->rear+1
- C. Q->front == (Q->rear+1)%M
- D. Q->front == (Q->rear-1)%M

答案：C · 考点 2.3 栈和队列

解析：逐项看 —— A 是队空条件，不是队满。

选项	判定
A. <code>front == rear</code>	✗ 这是队空条件
B. <code>front == rear+1</code>	✗ 少了取模 <code>%M</code> ，只在 <code>rear</code> 还没绕回数组末尾时偶然成立
C. <code>front == (rear+1)%M</code>	✓ 队满条件 —— <code>rear</code> 再走一格（取模绕回）就撞上 <code>front</code>
D. <code>front == (rear-1)%M</code>	✗ 符号反了（ <code>rear-1</code> 是往回走）

为什么队满条件是「rear 加 1 再取模」：

- 队列是环：`rear` 走到数组末尾后，`(rear+1)%M` 会绕回下标 0。
- 约定 `rear` 指向队尾元素的下一格（空位）。所以「`rear` 的下一格 = `front`」= 再存一个就会撞上队头 → 满。
- 代价：这套判据牺牲一个存储空间（严蔚敏版教材的做法）。所以长度为 M 的数组最多存 M-1 个元素。
- 为什么不直接用 `front == rear` 判满：队空时 `front == rear` 也成立 —— 同一个条件对应两种状态，无法区分。

循环队列三个公式（必背）：

条件	公式	说明
队空	<code>front == rear</code>	两个指针重合
队满	<code>(rear + 1) % M == front</code>	牺牲一个存储空间
元素个数	<code>(rear - front + M) % M</code>	先减后加 M 再取模，防负数

队列基础（本题解析里复习到的）：

队列：操作受限的线性表 —— 一端插入（队尾 `rear`）、另一端删除（队头 `front`），先进先出 FIFO。插入叫入队，删除叫出队。

指针	指向
front	队头元素
rear	队尾元素的下一个位置 ← 注意不是队尾元素本身

操作	步骤
入队	先送值到队尾位置，再 rear + 1
出队	先取队头元素，再 front + 1

⚠ 本页只考**顺序实现**（不考链式）。

📌 见 [2.3 栈和队列](#)。

二、判断题（本大题共 10 小题，每小题 2 分，共 20 分）

判断正确填「√」， 错误填「×」。

第 21 题 • 空格串与空串

空格串简称空串。（ ）

答案与解析

答案：× · 考点 **2.4 串、数组和广义表**

解析：

- **空格串** = 由一个或多个空格字符组成的串，长度 ≥ 1 ，例如 " "（3 个空格）长度是 3。

- **空串** = 长度为 **0** 的串，写作 `""`，里面一个字符都没有。
- 两者是**不同概念**：`strlen("") = 0`，`strlen(" ") = 1`。
- 一句话记牢：**空串没有字符，空格串有字符（只是恰好都是空格）。**

第 22 题 · 时间复杂度

程序段 `{n++;s+=n;}` 的时间复杂度为 $O(n)$ 。（ ）

答案与解析

答案：× · 考点 2.9 算法基本概念与分析

解析：

- 程序段里只有**两条语句**（`n++` 和 `s+=n`），**各执行一次就结束了**。
- 执行次数是**常数次**（2 次）→ 时间复杂度是 **$O(1)$** ，不是 $O(n)$ 。
- **核心判据**：时间复杂度看的是「**执行次数随 n 增长的阶**」，**不是**「代码里有没有出现变量 n 」。
- 常见陷阱：`n` 出现了不等于 $O(n)$ 。要看 `n` 是否**出现在循环条件/循环次数里**。

📌 时间复杂度「三步法」（P3 讲解，本卷必考技能）

很多同学靠「凭感觉」猜复杂度 —— 这个方法把猜测变成**机械推导**：

步	做什么
①	列表格 ：循环次数从 0 开始列（0、1、2...k），写出每轮之后 循环条件里那个量 变成了多少
②	写结束条件 ：从表格找规律，写出「第 k 次时条件不成立」的式子，再令它 等于 n （约等）
③	解方程 ：解出 k，k 的表达式就是复杂度（去掉系数、只留最高阶）

三个必会例子（都出自本卷讲解）：

循环	次数表	结束条件	解出 k	复杂度
<code>while (i*i*i <= n) i++;</code>	0 次→1 ³ , 1 次→2 ³ , ..., k 次→k ³	k ³ > n ⇒ k ³ = n	k = $\sqrt[3]{n}$	O($\sqrt[3]{n}$)
<code>while (i <= n) i *= 2;</code>	0 次→1, 1 次→2, 2 次→2 ² , ..., k 次→2 ^k	2 ^k > n ⇒ 2 ^k = n	k = log ₂ n	O(log n) (底数省略)
双层 for (外层 i=1...n, 内层 j≤2i)	内层次数 = 2、4、6、...、2n	—	等差数列求和	O(n ²)

两条收尾规则：

1. 只留最高阶： `0(n² + n)` → **O(n²)**; `0(3n)` → **O(n)** (系数扔掉) 。
2. 对数省略底数： `0(log2n)` = `0(log n)` 。因为 n 趋于无穷，换底只差一个常数系数。

双层循环速算：把「外层每个 i 值对应的内层次数」列出来，**求和**，再化简。上例： `2+4+6+...+2n` = 等差数列 = `n(2+2n)/2` = `n²+n` → **O(n²)**。

第 23 题 • printf 与 '\0'

C语言中，printf函数输出字符串时，遇到'\0'就停止输出。（)

答案与解析

答案：√ · 考点 1.6 数组

解析：

- `'\0'` 是 **ASCII 值为 0** 的字符，是 C 语言约定的**字符串结束标记**（NUL）。
- `printf("%s", p)` 从 `p` 开始逐字符输出，**遇到第一个 `'\0'` 立即停止**。
- **验证：**`printf("%s", "ab\0cd")` 只输出 `ab`，后面的 `cd` 不会被输出。
- 配套记忆：`char s[5] = "abc";` 实际存的是 `'a', 'b', 'c', '\0', '\0'`（数组长度 5，但**字符串长度 3**）。

第 24 题 · 数组名作实参

C语言中，若用数组名作为调用函数的实参，则传递给形参的数组的第一个元素值。（ ）

答案与解析

答案：× · 考点 1.8 指针

解析：

- 数组名作实参时，传递的是**数组首元素的地址**（即指针），**不是**首元素的值。
- 等价关系：`a` $\Leftrightarrow$ `&a[0]`（数组名本身是**地址常量**）。
- 所以形参要写成**指针或数组形式**：`void fun(int a[])` $\Leftrightarrow$ `void fun(int *a)`。
- **为什么这条很重要**：正因为传的是**地址**，函数内 `a[i] = x` 才会**真正改到主调函数的数组**；如果传的是「值」（拷贝），改不动。
- 反例记忆：数组名**不能被赋值**（`a = p;` 报错），因为它是**地址常量**；但 `int *p = a;` 合法。

第 25 题 · 完全二叉树

一棵完全二叉树有31个结点，则该二叉树是满二叉树。（ ）

答案与解析

答案：√ · 考点 2.5 树和二叉树

解析：

- **满二叉树**结点数公式：`n = 2h - 1`（h 为层数）。
- 代入：`31 = 25 - 1` ☒ —— 所以它正好是**5 层满二叉树**。

- **判定规律**：完全二叉树的结点数**恰好等于** $2^h - 1$ 时，它**必然是满二叉树**（因为完全二叉树是「从上到下、从左到右」依次排列，排满 $2^h - 1$ 个结点就一层不差）。
 - **对比记忆**：
 - 满二叉树：每层都满，结点数 **必为** $2^h - 1$ 。
 - 完全二叉树：只允许**最后一层**从右往左缺结点；结点数**不一定是** $2^h - 1$ 。
 - 31 是 $2^5 - 1 \rightarrow$ 判「满」；若是 30 个结点，就**不是**满二叉树（仍是完全二叉树）。
-

第 26 题 • 语句跨行

一个C语言程序的一个语句可以分写在多行。（ ）

答案与解析

答案：√ · 考点 1.1 C语言概述与基本概念

解析：

- C 语言**以分号 ; 作为语句的结束符**，换行符只是**空白字符**，仅起分隔作用。
- 所以一条语句可以分写多行：

```
int sum = a + b
        + c
        + d;    /* 合法：分号之前都算同一条语句 */
```

- 反过来也成立：**多条语句可以写在同一行**（`a=1; b=2; c=3;` 合法）。
 - ⚠ **不要混淆**：预处理指令（`#include`、`#define`）**默认以换行结束**，分行要用**续行符 ** —— 那是另一条规则。
-

第 27 题 · 栈的出栈序列

一个空栈S依次全部压入字符c、i、g、a、m后，再进行出栈至S为空，得到字符串为"ciugam"。（ ）

答案与解析

答案：× · 考点 2.3 栈和队列

解析：

- 栈的特点是 **LIFO（后进先出）**。入栈顺序 $c \rightarrow i \rightarrow g \rightarrow a \rightarrow m$ ，**m 在栈顶**。
- 「一次全部压入，再出栈」= 把整个序列**逆序**弹出： $m \rightarrow a \rightarrow g \rightarrow i \rightarrow c \rightarrow$ 得到 **"magic"**。
- 题面说得到 **"ciugam"** —— 那是**入栈顺序（原序）**，**错误**。
- 一句话口诀：**全部进栈再出栈 = 原序列倒过来**。

🔗 栈的出栈序列：本卷讲了 3 种考法（P3）

考法	规则	例（入栈 A B C D E）
① 全进再出	出栈序列 = 入栈序列 逆序	E D C B A
② 边进边出	进一个出一个 → 出栈序列 可以等于 入栈序列	A B C D E
③ 判「不属于」	给一个序列问「是否可能」→ 逐个模拟	见下

③ 的判定铁律：出栈的元素只能来自「当前栈顶」或「尚未入栈的后续元素」。

- 反例：入栈 **A B C D E**，出栈序列写 **C A ...** → **不可能**。因为 C 出栈后，栈里剩 **A B**，**B 在 A 上面**，下一个只能是 **B**，A 要等 B 走了才能走。
- 附加考点：问「这种进出栈序列**需要几个存储空间**」。若进出序列相同（边进边出），只需 **1 个**存储空间。

第 28 题 · 链式存储单元

线性表采用链式存储时，其存储单元必须是不能连续的。（）

答案与解析

答案：× · 考点 2.2 线性表

解析：

- 两种存储方式对物理连续性的要求完全不同：

存储方式	存储单元是否必须连续	靠什么找下一个元素
顺序存储（顺序表）	必须连续	下标运算（随机访问）
链式存储（链表）	可连续，也可不连续	结点里的指针域

- 题面说「必须不能连续」→ 错。链表靠指针相连，不靠物理相邻，所以连不连续都行。
- 一句话记牢：只有顺序存储强制连续；链式存储对连续性没有要求。

第 29 题 · 函数声明

double fun(int x,int y)是正确的函数声明。（）

答案与解析

答案：√ · 考点 1.7 函数

解析：

- 函数声明（原型）的形参列表里，每个形参都必须单独写出类型。
- double fun(int x, int y) —— x 是 int，y 也是 int，合法 ✓
- 陷阱形式（必须判错）：double fun(int x, y) ✗ —— 这里的 y 没有类型。

- 为什么容易错：定义普通变量时 `int x, y;` 是合法的（同一类型可连写），但形参列表里不允许这样简写。

- 补充：函数声明只需写类型也合法：`double fun(int, int);` —— 形参名可省略。
- 返回值类型 `double` 写在最前面，与形参类型互不影响。

第 30 题 • do-while 特性

C语言中，do-while的循环体至少无条件执行一次。（ ）

答案与解析

答案：√ · 考点 1.5 循环结构程序设计

解析：

- `do-while` 是直到型循环：先执行循环体，再判断条件。所以循环体至少执行 1 次。

```
do {  
    循环体;  
} while (条件);    /* ← 注意结尾的分号，不能少 */
```

- 对比 `while`（当型循环）：先判断，后执行。条件一开始就不成立 → 循环体执行 0 次。

```
while (条件) {  
    循环体;  
}
```

- 口诀：`do-while` 至少 1 次，`while` 可以 0 次。
- ⚠ 易错点：`do-while` 的 `while(条件);` 结尾分号必须写，漏了就语法错。

三、填空题（本大题共 5 小题，每小题 4 分，共 20 分）

第 31 题 • 标准化指数形式

C语言中，实数12345.6的标准化指数形式是____

答案与解析

答案： 0.123456e5 · 考点 1.2 数据的存储与运算

解析：

- 「标准化指数形式」的教材定义（两个条件）：
 - 小数点前的数字是 0；
 - 小数点后第一位数字不是 0。
- 按定义把 12345.6 改写成 0.123456 —— 小数点从 12345.6 里的位置向左移动了 5 位。
- 所以乘 10^5 ： $12345.6 = 0.123456 \times 10^5$
- C 语言的写法用 e 表示 10 的幂： 0.123456e5

⚠ 别和「规格化」搞混：

写法	是否符合「标准化」	说明
0.123456e5	✓	小数点前是 0，后第一位 $2 \neq 0$ —— 本题答案
1.23456e4	✗	小数点前是 1（不是 0） —— 这是「规格化」，不是教材说的标准化
12.3456e3	✗	小数点前有两位

🔴 注意：视频中该题答案数字被 UP 主打了马赛克（只留一个红勾），本页答案由逐字稿讲解还原（老师原话：「写成 0.123456 对吧，小数点往前移了 5 位，所以是乘以 10 的 5 次方」）。

第 32 题 · 二叉树结点关系

在一棵二叉树中，度为0的结点个数为 n_0 ，度为2的结点数为 n_2 ，则有 $n_0 - n_2 =$ ____

答案与解析

答案：1 · 考点 2.5 树和二叉树

解析：

- 这是二叉树的**核心性质**： $n_0 = n_2 + 1$ （叶子结点数 = 度为 2 的结点数 + 1），所以 $n_0 - n_2 = 1$ 。
- 「度」的定义：某结点有几个**子结点**，它的度就是几。
- 有两个子结点 → 度为 2；只有一个 → 度为 1；没有子结点（叶子）→ 度为 0。

推导（两条式子联立）：

式子	内容
① 结点总数	$n = n_0 + n_1 + n_2$
② 边数	从下看：每个结点（除根）对应一条边 → $边数 = n - 1$ ；从上数：每个结点的出边数之和 → $边数 = 0 \cdot n_0 + 1 \cdot n_1 + 2 \cdot n_2$
联立	$n - 1 = n_1 + 2n_2$ ，代入 ① → $n_0 + n_1 + n_2 - 1 = n_1 + 2n_2 \rightarrow n_0 = n_2 + 1$

画图验证（老师用的方法，最快）：画 A 带两个子结点 B、C（B、C 都是叶子）

- $n_0 = 2$ (B、C) · $n_2 = 1$ (A) → $2 - 1 = 1$ ✓

✦ 记法：叶子数 = 双分支结点数 + 1。这个式子与 n_1 （度为 1 的结点数）无关， n_1 是干扰项。

第 33 题 · 逻辑运算求值

执行C程序段 `int x = 1, y = 2, z = 0, t; t = (x && y) < (y || z);` 则t的值为____

答案与解析

答案：0 · 考点 1.2 数据的存储与运算

解析：

第 1 步 —— 算括号里的逻辑与 `(x && y)`：

- `x = 1` (非 0 → 真) , `y = 2` (非 0 → 真)
- 逻辑与：两边都真才真 → 结果真，C 语言里「真」的值为 1

第 2 步 —— 算括号里的逻辑或 `(y || z)`：

- `y = 2` (真) , `z = 0` (假)
- 逻辑或：有一个真就是真 → 结果真，值为 1

第 3 步 —— 算关系运算 `<`：

- `1 < 1` → 不成立 → 关系表达式的值为 0
- 赋给 `t` → `t = 0`

本题的两个考点：

考点	内容	
逻辑运算的结果值	<code>&&</code> 、 <code> </code>	<code> </code> 的结果不是两边的原值，而是 1 (真) 或 0 (假)
括号优先级最高	有括号先算括号，不要被 <code><</code> 与 <code>&&</code> / <code> </code>	<code> </code> 的优先级顺序干扰

💡 顺便记住优先级链（本卷单选第 14 题也考了）：

括号 → 单目 → 算术 → 关系 → 逻辑与 && → 逻辑或 || → 条件 → 赋值 → 逗号

即：< 的优先级高于 && 和 ||。本题**因为有括号**，才先算逻辑运算——若写成 x && y < y || z，结果就完全不同了。

第 34 题 • fclose 返回值

C语言fclose函数成功执行关闭操作后，返回值为____

答案与解析

答案：0 · 考点 1.10 文件操作

解析：

- fclose(fp) 关闭成功 → 返回 0；失败 → 返回非 0（通常是 EOF，即 -1）。

文件操作函数的返回值对照表（高频易混）：

函数	成功返回	失败返回
fopen	文件指针	NULL（空指针）
fclose	0	非 0（EOF / -1）
fgetc	读到的字符	EOF
fputc	写入的字符	EOF
fread / fwrite	实际读/写的数据项个数	小于要求个数
fscanf	成功赋值的变量个数	EOF

- 规律小结：「关闭成功返回 0」（与 `main` 返回 0 表示正常结束一致）；「打开成功返回指针」；「读写失败返回 EOF」。
- 标准写法：

```
FILE *fp = fopen("a.txt", "r");  
if (fp == NULL) { printf("打开失败\n"); return 1; }  
/* ... 读写 ... */  
fclose(fp);      /* 成功返回 0 */
```

🌸 视频中老师说这块**原本打算单独录一个文件操作专题**，本卷只讲了 `fclose` 的返回值这一个点。

第 35 题 • 十进制转八进制

十进制数 $(1348)_{10}$ 转换为八进制数为_____

答案与解析

答案： $(2504)_8$ · 考点 1.2 数据的存储与运算

解析：

十进制 → 其他进制：除基取余，倒序写出（除到商为 0 为止）

```
1348 ÷ 8 = 168 余 4  ↑  
168 ÷ 8 = 21 余 0  ↑  
21 ÷ 8 = 2 余 5  ↑  
2 ÷ 8 = 0 余 2  ↑
```

余数倒着读（从下往上）：2 5 0 4 → $(2504)_8$

验算（用「按权求和」反过来算）：

$$\begin{aligned} &2\times8^3 + 5\times8^2 + 0\times8^1 + 4\times8^0 \\ &= 2\times512 + 5\times64 + 0 + 4 \\ &= 1024 + 320 + 4 \\ &= 1348 \quad \checkmark \end{aligned}$$

13 进制转换两个方向（P4 讲解）

方向	方法	例子
十进制 → R 进制	除 R 取余，倒序写出（商为 0 停止）	$15_{10} \rightarrow 1111_2$; $15_{10} \rightarrow 17_8$
R 进制 → 十进制	按权展开求和（位权 = R 的「位序号」次方，从右往左第 0 位起）	$(1111)_2 = 1\times2^3+1\times2^2+1\times2^1+1\times2^0 = 15$

例： $(15)_{10} \rightarrow$ 二进制

$$\begin{aligned} 15 \div 2 &= 7 \text{ 余 } 1 \quad \uparrow \\ 7 \div 2 &= 3 \text{ 余 } 1 \quad \uparrow \\ 3 \div 2 &= 1 \text{ 余 } 1 \quad \uparrow \\ 1 \div 2 &= 0 \text{ 余 } 1 \quad \uparrow \end{aligned}$$

倒序读 → 1111；反算 $8+4+2+1 = 15$ ✓

⚠ 三条铁律（老师专门强调）：

- 1. 八进制里没有数字 8（只有 0-7）。
- 2. 十六进制里没有 G（只有 0-9 和 A-F，大小写都可以）。
- 3. 位序号从右往左从 0 开始数 —— 这是「按权求和」最容易数错的地方。

四~六、简答题 / 计算题 / 应用题

● 这三大类共 100 分，本视频合集没有覆盖。

依据：P4 结尾老师原话 ——「那咱们的选择题就给大家讲完了」，与 P1 开头「先看第一部分，单项选择题」呼应。

该合集（P1–P4）只讲解**客观题**：单项选择题（1–20）、判断题（21–30）、填空题（31–35），合计 **100 分**。

| 题型 | 分值 | 本页状态 |

| :--- | :--- | :--- |

| 四、简答题（4 题） | 40 | ✖ 无视频源 |

| 五、计算题（3 题） | 30 | ✖ 无视频源 |

| 六、应用题（3 题） | 30 | ✖ 无视频源 |

替代练法（不要空着）：

- 同型题 → [2025 真题回忆版](#)（含简答/计算/应用三类重构题）

- 历年真卷 → [2024 全卷](#)（四~六题型齐全，有完整答案与解析）

- 编程题专项 → [3.3 编程题做题策略](#) · [3.4 循环与数组综合编程专项](#) · [3.5 指针专项](#) · [3.6 递归与函数设计专项](#)

- 改错题（2026 新题型） → [3.0 改错题专项训练](#)

还原说明（方法可复核）

项	说明
源	B站 BV1Z4w3znE6h 「广东省计算机专升本真题讲解（历年真题合集）」P1–P4（UP：一灯computer）

P1	1710s → 第 1–4 题（单选）
P2	2507s → 第 5–20 题（单选）
P3	2019s → 第 21–30 题（判断题）+ 循环队列三公式 + 时间复杂度三步法
P4	896s → 第 31–35 题（填空题）+ 进制转换两方向
方法	逐字稿（ASR）+ 画面帧逐题核对。 选项与答案以画面帧为准
已修正的 ASR 失真	单选：checkGPT → chatgpt · depthsick/dpck → deepseek · out2 → 关键字（疑似 auto） 判断/填空：阎蔚敏 → 严蔚敏（教材作者）· real / font → rear/front（队列指针）· method size → MaxSize · 成全求和 → 按权求和 · CIGAM → 题面实为小写 c i g a m
已知缺口	① 第 19、20 题题面原文未抽帧捕获（老师中途切了画面） ② 第 31 题答案数字在视频中被 UP 主打打了马赛克（只留红勾），本页答案由逐字稿讲解还原
书写差异	第 17 题的数组初始化在原卷是两个连续左花括号开头，本页写成 { {0, 1, 2}, ...}（中间多一个空格）—— 因为本站是 VitePress 站，连续双花括号会被当成 Vue 插值导致构建失败。C 语言忽略空白，语义完全相同。改本页时别把这个空格去掉

● 答案的可靠性（分级标注，不混用）：

| 范围 | 可靠性 | 依据 |

|:---|:---|:---|

| 第 1–20 题（单选） | ✓ 题面、选项、答案三重一致 | 帧上有原文 + 帧上有答案标记 + 讲解印证 |

| ~~第 19、20 题~~ | ✓ 09-18 已补齐 | 对 P2 末尾 33:00–41:50 补抽 53 帧后捕获到完整题面与选项 |

| 第 21–30 题（判断） | ✓ 题面 + 答案一致 | 判断题在帧上有完整题干（21–30 同一屏），答案由讲解逐题印证 |

| 第 31–35 题（填空） | ✓ 题面一致，第 31 题答案为讲解还原 | 题干在帧上；第 31 题答案被马赛克遮挡 |

本页已还原 100 分（客观题全部，题面无缺口）。四~六题型（100 分）无视频源，见上方说明。